Problem 1

Problem Statement

Develop a rule-based expert system that accepts symptoms from the user and applies a predefined set of **IF–THEN inference rules** to diagnose a possible medical condition. The system should simulate the decision-making process of a medical expert by matching symptoms against rules stored in a knowledge base.

Knowledge Representation (Sample Rules)

Rule No.	IF (Symptoms)	THEN (Diagnosis)
R1	IF body_temperature > 38°C AND headache	THEN Fever
R2	IF sneezing AND runny_nose AND sore_throat	THEN Common Cold
R3	IF high_fever AND body_ache AND fatigue	THEN Flu
R4	IF cough AND mild_fever	THEN Common Cold
R5	IF chills AND high_fever	THEN Flu

Inference Mechanism

- **Input:** Symptoms provided by the user
- **Processing:** Forward chaining rule matching
- **Output:** Diagnosed disease based on matched rules

Example Case

User Inputs:

- Body Temperature: 39°C
- Symptoms: Body ache, fatigue, chills

Rule Fired:

- R3: IF high_fever AND body_ache AND fatigue → Flu
- R5: IF chills AND high_fever → Flu

System Output:

Diagnosis: Flu

Expected Output

The system should display:

- Identified disease
- Matched rule(s)
- Advisory message (e.g., consult a doctor)

Problem 2

To implement **Breadth First Search (BFS)** and **Depth First Search (DFS)** algorithms and analyze their traversal behavior on a larger graph.

Example Graph

Consider the following **undirected graph** with **9 nodes**:

```
          A
        /   \
       B     C
      / \   / \
     D   E F   G
         |     |
         H     I
```

Vertices

A, B, C, D, E, F, G, H, I

Edges

(A–B), (A–C),
(B–D), (B–E),
(C–F), (C–G),
(E–H), (G–I)

Starting Node

A

Adjacency List Representation

```
A → B, C
B → A, D, E
C → A, F, G
D → B
E → B, H
F → C
G → C, I
H → E
I → G
```

Breadth First Search (BFS)

Traversal Logic

- Start from node **A**
- Visit all neighbouring nodes first
- Use a **queue**

BFS Traversal Order

A → B → C → D → E → F → G → H → I

Depth First Search (DFS)

Traversal Logic

- Start from node **A**
- Go as deep as possible before backtracking
- Use **recursion or stack**

DFS Traversal Order

A → B → D → E → H → C → F → G → I

Expected Output

The program should display:

```
BFS Traversal: A B C D E F G H I
DFS Traversal: A B D E H C F G I
```